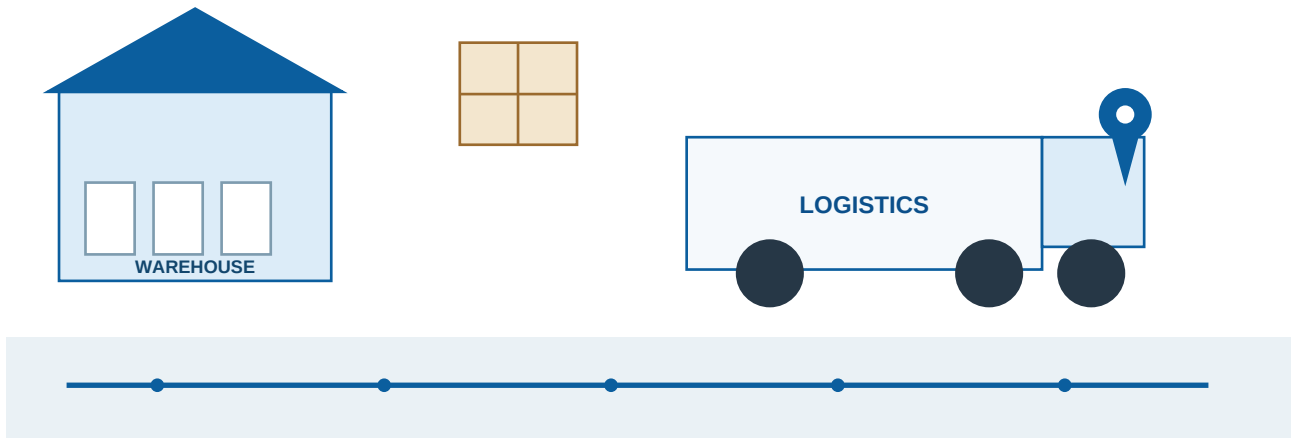


# LOGISTICS MANAGEMENT SYSTEM

REST API — PROJECT DOCUMENTATION



**Project Name:** Logistics Management System

**Technology Stack:** Java 26 • Spring Boot 4.1.1 • Spring Data JPA • MySQL • Maven • Lombok • Jakarta Validation • Spring Security • JWT • AOP • Swagger/OpenAPI • Cache • Actuator • Postman

Customer Management • Warehouse & Inventory • Shipment Tracking • Delivery Management

# 1. Project Overview

The Logistics Management System REST API is a backend application developed using Spring Boot. The project manages customers, warehouses, inventory, shipments, tracking updates, and deliveries through RESTful APIs. The uploaded project also contains JWT authentication, Spring Security, validation, global exception handling, AOP logging/auditing, scheduled shipment-status processing, caching configuration, Swagger/OpenAPI configuration, file upload/download, a dashboard, and a custom Actuator logistics endpoint.

The application follows a layered architecture consisting of Controller, Service, ServiceImplementation, Repository, Entity, DTO, Mapper, Security, Exception, Scheduler, Configuration, AOP, and Actuator components.

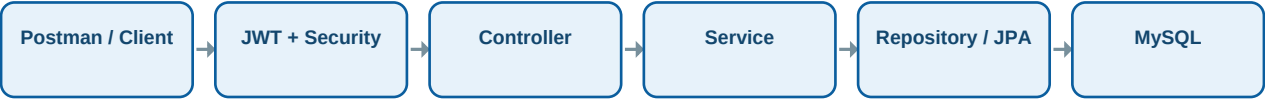
# 2. Objectives

- Manage customer information and customer shipments.
- Create and manage warehouses and their inventory.
- Create shipments with origin, destination, and status.
- Maintain shipment tracking history with locations and timestamps.
- Manage delivery records associated with shipments.
- Provide CRUD REST APIs for the main logistics resources.
- Use DTOs and mapper classes to separate API models from JPA entities.
- Validate incoming request data with Jakarta Validation.
- Handle errors through centralized exception handling.
- Secure protected APIs using JWT authentication and Spring Security.
- Support pagination/sorting for customer records and filtered shipment queries.
- Provide operational visibility through dashboard and Actuator endpoints.

# 3. Technologies Used

| Technology               | Purpose                                                 |
|--------------------------|---------------------------------------------------------|
| Java 26                  | Programming Language                                    |
| Spring Boot 4.1.1        | Backend Framework                                       |
| Spring Data JPA          | ORM and Database Operations                             |
| MySQL                    | Relational Database                                     |
| Maven                    | Dependency and Build Management                         |
| Lombok                   | Reduce Boilerplate Code                                 |
| Jakarta Validation       | Request Validation                                      |
| Spring Security          | Authentication and Authorization                        |
| JWT 0.12.6               | JWT Token Processing                                    |
| Spring AOP / AspectJ     | Logging, auditing, execution-time and exception aspects |
| Springdoc OpenAPI 2.8.13 | Swagger/OpenAPI Documentation                           |
| Spring Cache             | Caching Support                                         |
| Spring Actuator          | Application Monitoring and Custom Endpoint              |
| Postman                  | API Testing                                             |

# 4. Project Architecture



## 5. Project Structure

```
com.example
├── Actuator
├── LogisticsEndPoint
├── Aspect
│   ├── LoggingAspect
│   ├── AuditAspect
│   ├── ExecutionTimeAspect
│   └── ExceptionLoggingAspects
├── Auth
│   ├── AuthController / AuthService / AuthServiceImpl
│   ├── JwtService
│   ├── LoginRequest / RegisterRequest
│   └── AuthResponse
├── Config
│   ├── CacheConfig
│   ├── JPAConfig
│   └── OpenApiConfig
├── Controller
│   ├── Customer / Warehouse / Inventory
│   ├── Shipment / Tracking / Delivery
│   └── Dashboard / File
├── DTO
├── Entity
├── Exception
├── Mapper
├── Repository
├── Scheduler
├── Security
├── Service
├── ServiceImplementation
└── LogisticsManagementSystemApplication
```

The project also contains automated tests for the application, customer controller, customer repository, customer service, logging aspect, and security.

## 6. Database Design

| Entity    | Main Fields                                                                  | JPA Relationship                                                |
|-----------|------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Customer  | customerId, customerName, email, phoneNumber, address, createdAt, modifiedAt | One-to-Many → Shipment                                          |
| Warehouse | id, name, location, capacity                                                 | One-to-Many → Inventory                                         |
| Inventory | id, warehouse, ProductName, Quantity, lastUpdated                            | Many-to-One → Warehouse                                         |
| Shipment  | id, customer, origin, destination, status, shipmentDate                      | Many-to-One Customer; One-to-Many Tracking; One-to-One Delivery |
| Tracking  | id, shipment, location, status, timestamp                                    | Many-to-One → Shipment                                          |
| Delivery  | id, shipment, deliveryDate, deliveryAddress, deliveredBy, status             | One-to-One → Shipment                                           |
| User      | id, username, password, role                                                 | Used by JWT/Spring Security                                     |

Development profile uses the MySQL database **logisticsmanagementsystem** with JPA **ddl-auto=update**. The production profile is configured with **ddl-auto=validate**.

## 7. Entity Relationships



### Relationship details:

- One Customer can have many Shipments.
- One Warehouse can have many Inventory records.
- One Shipment can have many Tracking updates.
- One Shipment can have one Delivery record; the delivery shipment foreign key is unique.
- Tracking and Delivery records point back to their Shipment.

## 8. API Endpoints

### Authentication APIs

| Method | Endpoint       | Description                           |
|--------|----------------|---------------------------------------|
| POST   | /auth/register | Register a user                       |
| POST   | /auth/login    | Authenticate and receive JWT response |

### Customer APIs

| Method | Endpoint                                                      | Description            |
|--------|---------------------------------------------------------------|------------------------|
| POST   | /customers                                                    | Create customer        |
| GET    | /customers                                                    | Get all customers      |
| GET    | /customers/{id}                                               | Get customer by ID     |
| GET    | /customers/page?page=0&size=5&sortBy=customerId&direction=asc | Pagination and sorting |
| PUT    | /customers/{id}                                               | Update customer        |
| DELETE | /customers/{id}                                               | Delete customer        |

### Warehouse / Inventory APIs

| Method | Endpoint                           | Description                |
|--------|------------------------------------|----------------------------|
| POST   | /warehouses                        | Create warehouse           |
| GET    | /warehouses                        | Get all warehouses         |
| GET    | /warehouses/{id}                   | Get warehouse by ID        |
| PUT    | /warehouses/{id}                   | Update warehouse           |
| DELETE | /warehouses/{id}                   | Delete warehouse           |
| POST   | /inventory                         | Create inventory           |
| GET    | /inventory                         | Get all inventory          |
| GET    | /inventory/{id}                    | Get inventory by ID        |
| GET    | /inventory/warehouse/{warehouseId} | Get inventory by warehouse |

| Method | Endpoint        | Description      |
|--------|-----------------|------------------|
| PUT    | /inventory/{id} | Update inventory |
| DELETE | /inventory/{id} | Delete inventory |

## 9. API Endpoints — Shipment, Tracking and Delivery

| Method | Endpoint                          | Description                |
|--------|-----------------------------------|----------------------------|
| POST   | /shipments                        | Create shipment            |
| GET    | /shipments                        | Get all shipments          |
| GET    | /shipments/{id}                   | Get shipment by ID         |
| GET    | /shipments/customer/{customerId}  | Get shipments for customer |
| GET    | /shipments/status/{status}        | Get shipments by status    |
| PUT    | /shipments/{id}                   | Update shipment            |
| DELETE | /shipments/{id}                   | Delete shipment            |
| POST   | /tracking                         | Create tracking update     |
| GET    | /tracking                         | Get all tracking updates   |
| GET    | /tracking/{id}                    | Get tracking by ID         |
| GET    | /tracking/shipment/{shipmentId}   | Get tracking by shipment   |
| PUT    | /tracking/{id}                    | Update tracking            |
| DELETE | /tracking/{id}                    | Delete tracking            |
| POST   | /deliveries                       | Create delivery            |
| GET    | /deliveries                       | Get all deliveries         |
| GET    | /deliveries/{id}                  | Get delivery by ID         |
| GET    | /deliveries/shipment/{shipmentId} | Get delivery by shipment   |
| PUT    | /deliveries/{id}                  | Update delivery            |
| DELETE | /deliveries/{id}                  | Delete delivery            |

## 10. Additional APIs

| Method | Endpoint                   | Description                                                                               |
|--------|----------------------------|-------------------------------------------------------------------------------------------|
| GET    | /dashboard                 | Dashboard counts for customers, warehouses, inventory, shipments, tracking and deliveries |
| POST   | /files/upload              | Upload a multipart file                                                                   |
| GET    | /files/download/{fileName} | Download a stored file                                                                    |
| GET    | /actuator/health           | Spring Boot health information                                                            |
| GET    | /actuator/info             | Application information                                                                   |
| GET    | /actuator/metrics          | Application metrics                                                                       |
| GET    | /actuator/logistics        | Custom logistics status endpoint                                                          |

## 11. Validation Used

- @NotBlank — ensures required text fields are not empty.
- @Email — validates customer email format.
- @Pattern — validates the customer phone number using the configured 10-digit pattern.
- @Min — validates warehouse capacity and inventory quantity.
- @NotNull — validates required IDs in request DTOs.
- @Valid — triggers DTO validation in controller request bodies.

- `@Validated` — enables validation support at controller level.

Examples from the uploaded project include `CustomerRequest`, `WarehouseRequest`, `InventoryRequest`, `ShipmentRequest`, `TrackingRequest` and `DeliveryRequest`.

```
@NotBlank(message = "Customer name is required")
private String customerName;
```

```
@NotBlank(message = "Email is required")
@email(message = "Enter a valid email")
private String email;
```

```
@Pattern(regexp = "[6-9][0-9]{9}",
message = "Phone number must contain 10 digits")
private String phoneNumber;
```

## 12. Exception Handling

The project contains `ResourceNotFoundException`, `BadRequestException`, `ErrorResponse`, and `GlobalExceptionHandler`. Global exception handling is implemented with `@RestControllerAdvice` to return structured errors.

| Scenario             | Status                    | Purpose                                            |
|----------------------|---------------------------|----------------------------------------------------|
| Validation error     | 400 Bad Request           | Invalid request body or field value                |
| Bad request          | 400 Bad Request           | Application-level request problem                  |
| Resource not found   | 404 Not Found             | Requested customer/shipment/etc. does not exist    |
| Unauthorized         | 401 Unauthorized          | JWT is missing or invalid for a protected endpoint |
| Forbidden            | 403 Forbidden             | Authenticated user lacks required permission       |
| Unexpected exception | 500 Internal Server Error | Unhandled server-side error                        |

## 13. Security and JWT Authentication



- Register and login endpoints are configured as public endpoints.
- Other endpoints require authentication through the configured security filter chain.
- Passwords are processed using a `PasswordEncoder`.
- `JwtAuthenticationFilter` is added before `UsernamePasswordAuthenticationFilter`.
- `JwtService` generates, extracts, and validates JWT information.
- User details are loaded through `CustomUserDetailsService`.

Typical Postman header: **Authorization: Bearer <JWT\_TOKEN>**

## 14. AOP, Scheduling, Cache and Monitoring

### AOP Features

| Aspect                               | Purpose                                              |
|--------------------------------------|------------------------------------------------------|
| <code>LoggingAspect</code>           | Runs before and after service implementation methods |
| <code>AuditAspect</code>             | Logs successful method execution and returned result |
| <code>ExecutionTimeAspect</code>     | Measures service method execution time               |
| <code>ExceptionLoggingAspects</code> | Logs exceptions thrown by service methods            |

### Scheduling

ShipmentStatusScheduler runs every 60 seconds. It checks shipment dates and changes eligible shipments to **In Transit** when the shipment date is before the current time and the shipment is not already marked Delivered.

### Cache and Monitoring

- CacheConfig enables Spring caching with @EnableCaching.
- OpenApiConfig defines the Logistics Management System Swagger/OpenAPI metadata.
- Actuator exposes health, info, metrics and the custom logistics endpoint.
- The custom /actuator/logistics endpoint reports the application name and operational status.

## 15. Postman, Swagger and Sample Request

### Sample Customer Request

POST http://localhost:8080/customers  
Content-Type: application/json  
Authorization: Bearer

```
{
  "customerName": "Rajesh",
  "email": "rajesh@example.com",
  "phoneNumber": "9876543210",
  "address": "Madurai"
}
```

### Sample Customer Response

```
{
  "success": true,
  "message": "Customer Created Successfully...",
  "data": {
    "customerId": 1,
    "customerName": "Rajesh",
    "email": "rajesh@example.com",
    "phoneNumber": "9876543210",
    "address": "Madurai",
    "createdAt": "2026-09-16T10:30:45"
  },
  "timestamp": "2026-09-16T10:30:45"
}
```

### Swagger/OpenAPI

The uploaded project includes Springdoc OpenAPI and an OpenApiConfig class. When the application is running, the Swagger UI can be used to explore and test the REST endpoints exposed by the application.

| SWAGGER UI — LOGISTICS MANAGEMENT SYSTEM API        |
|-----------------------------------------------------|
| Authentication • Customers • Warehouses • Inventory |
| Shipments • Tracking • Deliveries • Dashboard       |
| Try it out → Execute → Inspect Response             |

## 16. Features

- Customer Management
- Warehouse Management
- Inventory Management
- Shipment Management
- Shipment Tracking
- Delivery Management
- Dashboard Statistics

- Pagination and Sorting for Customers
- Shipment Filtering by Customer and Status
- File Upload and Download
- DTO and Mapper Architecture
- Validation
- Global Exception Handling
- JWT Authentication and Spring Security
- AOP Logging, Auditing, Execution-Time and Exception Logging
- Scheduled Shipment Status Updates
- Caching Configuration
- Swagger/OpenAPI Documentation
- Actuator Monitoring
- Automated Tests

## 17. HTTP Status Codes

| Code                      | Description                     | Typical Use                          |
|---------------------------|---------------------------------|--------------------------------------|
| 200 OK                    | Successful request              | GET, PUT, successful DELETE response |
| 201 Created               | Resource created                | POST create operations               |
| 204 No Content            | Successful request without body | Optional REST response               |
| 400 Bad Request           | Invalid request                 | Validation or bad input              |
| 401 Unauthorized          | Authentication required/failed  | Missing or invalid JWT               |
| 403 Forbidden             | Access denied                   | Insufficient authorization           |
| 404 Not Found             | Resource unavailable            | Invalid ID                           |
| 500 Internal Server Error | Unexpected server failure       | Unhandled exception                  |

## 18. Future Enhancements

- Role-based access control with explicit business roles and permissions.
- Advanced shipment search, pagination and sorting across all resources.
- Inventory low-stock alerts and stock movement history.
- Real-time tracking integration with external location services.
- Email/SMS notifications for shipment and delivery status changes.
- More comprehensive unit, integration and security test coverage.
- Docker and cloud deployment.
- Centralized logging and monitoring dashboards.
- API versioning and expanded OpenAPI documentation.

## 19. Conclusion

The Logistics Management System REST API is a comprehensive Spring Boot backend that demonstrates practical logistics-domain development. Based on the uploaded source project, it combines customer, warehouse, inventory, shipment, tracking and delivery management with DTOs, JPA relationships, validation, global exception handling, JWT security, AOP, scheduling, caching, Swagger/OpenAPI, Actuator monitoring, file handling, dashboard statistics and automated tests. The layered design provides a clear foundation for maintenance, testing and future expansion.

